

Dharanidharan P

Test ID: 432008997521721 | 8870210964 | dharanidharanp.22cse@kongu.edu

Test Date: March 4, 2025

Computer Science 40 /100	Logical Ability 52 /100	Quantitative Ability (Advanced) 33 /100	English Comprehension 46 /100
WriteX - Essay Writing 5 /100	Automata 0 /100	Automata Fix 0 /100	Personality Completed

Computer Science			40 / 100
OS and Computer Architecture	DBMS	Computer Networks	
51 / 100	23 / 100	29 / 100	

Logical Ability			52 / 100
Inductive Reasoning	Deductive Reasoning	Abductive Reasoning	
55 / 100	52 / 100	49 / 100	

Quantitative Ability (Advanced)			33 / 100
Basic Mathematics	Advanced Mathematics	Applied Mathematics	
28 / 100	33 / 100	38 / 100	

English Comprehension



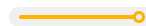
46 / 100

CEFR: **B1**

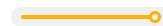
Grammar

Vocabulary

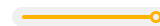
Comprehension



43 / 100



48 / 100



48 / 100

WriteX - Essay Writing



5 / 100

CEFR: **A1**

Content Score

Grammar Score



5 / 100



5 / 100

Automata



0 / 100

Programming Ability

Programming Practices

Functional Correctness



0 / 100



0 / 100



0 / 100

Automata Fix



0 / 100

Code Reuse

Syntactical Error

Logical Error



0 / 100



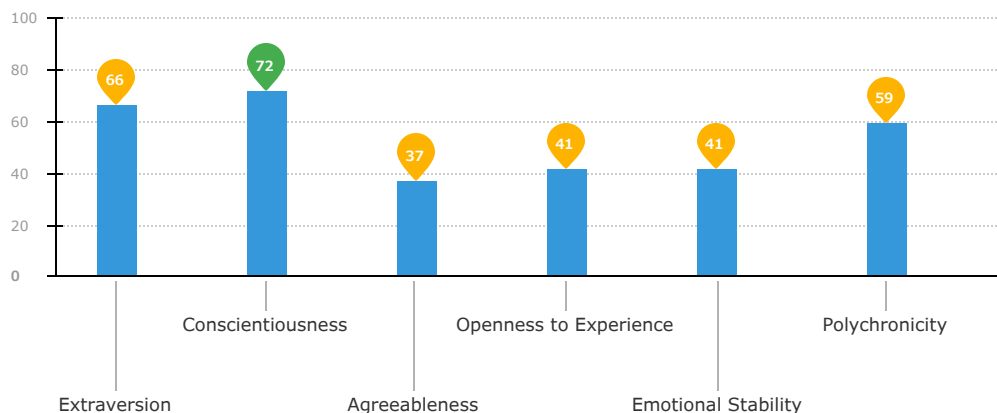
0 / 100



0 / 100

Personality

Completed



People Interaction



Self-Drive



Trainability



Repetitive Job Suitability



Competencies

Work attributes

1 | Introduction

About the Report

This report provides a detailed analysis of the candidate's performance on different assessments. The tests for this job role were decided based on job analysis, O*Net taxonomy mapping and/or criterion validity studies. The candidate's responses to these tests help construct a profile that reflects her/his likely performance level and achievement potential in the job role

This report has the following sections:

The **Summary** section provides an overall snapshot of the candidate's performance. It includes a graphical representation of the test scores and the subsection scores.

The **Insights** section provides detailed feedback on the candidate's performance in each of the tests. The descriptive feedback includes the competency definitions, the topics covered in the test, and a note on the level of the candidate's performance.

The **Response** section captures the response provided by the candidate. This section includes only those tests that require a subjective input from the candidate and are scored based on artificial intelligence and machine learning.

The **Learning Resources** section provides online and offline resources to improve the candidate's knowledge, abilities, and skills in the different areas on which s/he was evaluated.

Score Interpretation

All the test scores are on a scale of 0-100. All the tests except personality and behavioural evaluation provide absolute scores. The personality and behavioural tests provide a norm-referenced score and hence, are percentile scores. Throughout the report, the colour codes used are as follows:

- Scores between 67 and 100
- Scores between 33 and 67
- Scores between 0 and 33

2 | Insights

English Comprehension



46 / 100

CEFR: **B1**

This test aims to measure your vocabulary, grammar and reading comprehension skills.

You are able to state the meanings of English words used in day-to-day life. This is essential for you to read and understand short texts, write letters/emails, and understand internal communication at work.

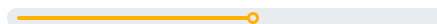
Logical Ability



52 / 100



Inductive Reasoning



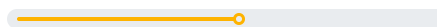
55 / 100

This competency aims to measure the your ability to synthesize information and derive conclusions.

You are able to work out rules based on specific information and solve general work problems using these rules. This skill is required in data-driven research jobs where one needs to formulate new rules based on variable trends.



Deductive Reasoning



52 / 100

This competency aims to measure the your ability to synthesize information and derive conclusions.

You are able to work out rules based on specific information and solve general work problems using these rules. This skill is required in data-driven research jobs where one needs to formulate new rules based on variable trends.



Abductive Reasoning



49 / 100

Quantitative Ability (Advanced)



33 / 100

This test aims to measure your ability to solve problems on basic arithmetic operations, probability, permutations and combinations, and other advanced concepts.

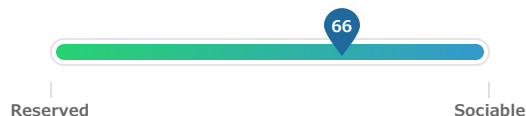
You are able to perform simple arithmetic operations. Apart from their relevance in monetary transactions, these operations are used in other situations, such as dividing up tasks with one's colleagues, managing one's time at work, and planning the resources required to complete a task.

Personality

Competencies



Extraversion



Extraversion refers to a person's inclination to prefer social interaction over spending time alone. Individuals with high levels of extraversion are perceived to be outgoing, warm and socially confident.

- You are comfortable socializing to a certain extent. You prefer small gatherings in familiar environments.
- You feel at ease interacting with your close friends but may be reserved among strangers.
- You indulge in activities involving thrill and excitement that are not too risky.
- You contemplate the consequences before expressing any opinion or taking an action.
- You take charge when the situation calls for it and you are comfortable following instructions as well.
- Your personality may be suitable for jobs demanding flexibility in terms of working well with a team as well as individually.



Conscientiousness

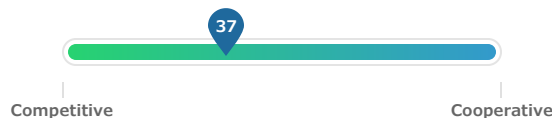


Conscientiousness is the tendency to be organized, hard working and responsible in one's approach to your work. Individuals with high levels of this personality trait are more likely to be ambitious and tend to be goal-oriented and focused.

- You value order and self discipline and tends to pursue ambitious endeavours.
- You believe in the importance of structure and is very well-organized.
- You carefully review facts before arriving at conclusions or making decisions based on them.
- You strictly adhere to rules and carefully consider the situation before making decisions.
- You tend to have a high level of self confidence and do not doubt your abilities.
- You generally set and work toward goals, try to exceed expectations and are likely to excel in most jobs, especially those which require careful or meticulous approach.



Agreeableness



Agreeableness refers to an individual's tendency to be cooperative with others and it defines your approach to interpersonal relationships. People with high levels of this personality trait tend to be more considerate of people around them and are more likely to work effectively in a team.

- You are flexible regarding your opinions and be willing to accommodate the needs of others.
- You are generally considerate of the needs of others yet may, at times, overlook social norms to achieve personal success.
- You are selective about the people you choose to trust.
- You are caring and you empathise a friend in distress.

- You give credit to others but also tends to be open with your friends about personal achievements.
- You are more inclined to strike a compromise in tough situations and may be suitable for jobs that demand managing expectations among different stakeholders.



Openness to Experience

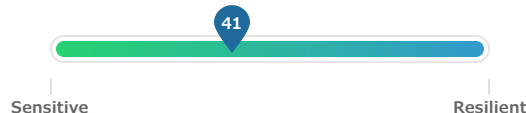


Openness to experience refers to a person's inclination to explore beyond conventional boundaries in different aspects of life. Individuals with high levels of this personality trait tend to be more curious, creative and innovative in nature.

- You may try new things but would prefer not to venture too far beyond your comfort zone.
- You tend to be open to accepting abstract ideas after weighing them against existing solutions.
- You appreciate the arts to a certain extent but may lack the curiosity to explore them in depth.
- You may express your feelings only to people you are comfortable with.
- Your personality is more suited for jobs involving a mix of logical and creative thinking.



Emotional Stability



Emotional stability refers to the ability to withstand stress, handle adversity, and remain calm and composed when working through challenging situations. People with high levels of this personality trait tend to be more in control of their emotions and are likely to perform consistently despite difficult or unfavourable conditions.

- You are calm and relaxed in most situations.
- You experience a range of emotions in high pressure situations. You tend to worry when working in critical conditions.
- You do not like attention drawn towards you. You take some time to become confident and comfortable around people.
- You subdue your impulses and tend to act in a rational manner.
- Your personality is suited for jobs that have a moderate amount of stress.



Polychronicity



Polychronicity refers to a person's inclination to multitask. It is the extent to which the person prefers to engage in more than one task at a time and believes that such an approach is highly productive. While this trait describes the personality disposition of a person to multitask, it does not gauge their ability to do so successfully.

- You neither have a strong preference nor dislike to perform multiple tasks simultaneously.
- You are open to both options - pursuing multiple tasks at the same time or working on a single project at a time.
- Whether or not you will succeed in a polychronous environment depends largely on your ability to do so.

3 | Response

Automata



0 / 100

[Code Replay](#)

Question 1 (Language: Python)

A company has a sales record of N products for M days. The company wants to know the maximum revenue received from a given product among the N products each day. Write an algorithm to find the highest revenue received each day.

Scores

Programming Ability



0 / 100

Programming ability score cannot be generated. This is because source code has syntax/ runtime errors and is unparseable.

Functional Correctness



0 / 100

Syntactically incorrect code. The source code has syntax errors in it.

Programming Practices



0 / 100

Programming practices score cannot be generated. This is because source code has syntax/runtime errors and is unparseable or the source code does not meet the minimum code-length specifications.

Final Code Submitted

Compilation Status: Fail

```

1
2 """
3
4 """
5 def maxRevenue(salesRecord):
6     # Write your code here
7     for(int i=0;i<n;i++):
8
9     return
10
11 def main():
12     # input for salesRecord
13     salesRecord = []
14     salesRecord_rows, salesRecord_cols = map(int, raw_input().split
15 ())
16     for idx in range(salesRecord_rows):
17         salesRecord.append(list(map(int, raw_input().split()))))
18
19 result = maxRevenue(salesRecord)
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code: $O(N \log N)$

*N represents days/products in the sales record.

Errors/Warnings

Compiling failed with exitcode 1, compiler output:
File "source_5887.py", line 8
for(int i=0;i ^
SyntaxError: invalid syntax

Structural Vulnerabilities and Errors

```

18
19 print(" ".join([str(res) for res in result]))
20
21 if __name__ == "__main__":
22     main()

```

There are no errors in the candidate's code.

Compilation Statistics



Total attempts



Successful



Compilation errors



Sample failed



Timed out



Runtime errors

Response time:

00:11:31

Average time taken between two compile attempts:

00:00:00

Average test case pass percentage per compile:

0%

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 2 (Language: Python)

An employee has to send a secret code S to their boss. They design a method to encrypt the code using two key values N and M . The formula that they use to develop the encrypted code is shown below:

$$(((S^N \% 10)^M) \% 1000000007)$$

Write an algorithm to help the employee encrypt the code.

Scores

Programming Ability

0 / 100

Programming ability score cannot be generated. This is because source code has syntax/ runtime errors and is unparseable.

Functional Correctness

0 / 100

Syntactically incorrect code. The source code has syntax errors in it.

Programming Practices

0 / 100

Programming practices score cannot be generated. This is because source code has syntax/runtime errors and is unparseable or the source code does not meet the minimum code-length specifications.

Final Code Submitted

Compilation Status: Fail

```

1
2 """
3
4 """
5 def findSecretCode(secretCode, firstKey, secondKey):
6     # Write your code here
7     for(int i=0;i<n;i++):
8         if(1<=secretCode):
9             print()
10
11     return
12
13 def main():
14     # input for secretCode
15     secretCode = int(raw_input())
16     # input for firstKey
17     firstKey = int(raw_input())
18     # input for secondKey
19     secondKey = int(raw_input())
20
21     result = findSecretCode(secretCode, firstKey, secondKey)
22     print result,
23
24 if __name__ == "__main__":
25     main()

```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code: $O(\log N)$

*N represents key value of N/M

Errors/Warnings

Compiling failed with exitcode 1, compiler output:
File "source_80.py", line 8
for(int i=0;i ^
SyntaxError: invalid syntax

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Compilation Statistics

0

Total attempts

0

Successful

0

Compilation errors

0

Sample failed

0

Timed out

0

Runtime errors

Response time:

00:28:25

Average time taken between two compile attempts:

00:00:00

Average test case pass percentage per compile:

0%

i Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

i Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Automata Fix



0 / 100

[Code Replay](#)

Question 1 (Language: Java)

The function/method **median** accepts two arguments - *size* and *inputList*, an integer representing the length of a list and a list of integers, respectively.

The function/method **median** is supposed to calculate and return an integer representing the median of elements in the input list. However, the function/method **median** works only for odd-length lists because of incomplete code.

You must complete the code to make it work for even-length lists as well. A couple of other functions/methods are available, which you are supposed to use inside the function/method **median** to complete the code.

Helper Description

The following class is used to represent a MedianCalculate and is already implemented in the default code (Do not write this definition again in your code):

```
class MedianCalculate
{
    public int[] items;

    public int first;

    public int last;

    public int order;

    int quick_select()
    {
        /*It calculate the median value

        This can be called as -

        MedianCalculate ob = new MedianCalculate(inputList, start_index, end_index, median_order);

        int res = ob.quick_select();

        where median_order is the half length of the inputList */

    }
}
```

Scores

Final Code Submitted

Compilation Status: Pass

```
1 // You can print the values to stdout for debugging
2 class Solution
3 {
4     float median(int size , int[] inputList)
5     {
6         int start_index = 0;
7         int end_index = size-1;
8         float res = -1;
9         if(size%2!=0) // odd length arrays
10        {
11            int median_order = ((size +1)/2);
12            MedianCalculate ob = new MedianCalculate(inputList, start_index, end_index, median_order);
13            res = ob.quick_select();
14        }
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilities and Errors

```

15     else // even length arrays
16     {
17         int median_order=((size+1)/2);
18         MedianCalculate ob=new MedianCalculate(inputList,start_in
dex,end_index,median_order);
19         res=ob.quick_select();
20     }
21     return res;
22 }
23 }

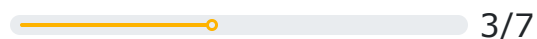
```

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **42.86%**

Total score



50%

Basic(2/4)

0%

Advance(0/2)

100%

Edge(1/1)

Compilation Statistics

7

Total attempts

7

Successful

0

Compilation errors

5

Sample failed

0

Timed out

2

Runtime errors

Response time:

00:06:01

Average time taken between two compile attempts:

00:00:52

Average test case pass percentage per compile:

14.3%

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 2 (Language: Java)

The function/method **multiplyNumber** returns an integer representing the multiplicative product of the maximum two of three input numbers. The function/method **multiplyNumber** accepts three integers- *numA*, *numB* and *numC*, representing the input numbers.

The function/method **multiplyNumber** compiles unsuccessfully due to syntactical error. Your task is to debug the code so that it passes all the test cases.

Scores

Final Code Submitted

Compilation Status: Fail

```

1 // You can print the values to stdout for debugging
2 class Solution
3 {
4     int multiplyNumber(int numA, int numB, int numC)
5     {
6         int result,min,max,mid;
7         max=(numA>numB)?numA>numC)?numA:numC):(numB>num
8 C)?numB:numC);
9         min=(numA<numB)?((numA<numC)?numA:numC):((numB<nu
10 mC)?numB:numC);
11         mid=(numA+numB+numC)-(min+max);
12         result=(max*int mid);
13     }
14 }
15

```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

Solution.java:7: error: : expected
max=(numA>numB)?numA>numC)?numA:numC):
(numB>numC)?numB:numC);
^

Solution.java:7: error: ';' expected
max=(numA>numB)?numA>numC)?numA:numC):
(numB>numC)?numB:numC);
^

Solution.java:7: error: not a statement
max=(numA>numB)?numA>numC)?numA:numC):
(numB>numC)?numB:numC);
^

Solution.java:7: error: ';' expected
max=(numA>numB)?numA>numC)?numA:numC):
(numB>numC)?numB:numC);
^

Solution.java:7: error: not a statement
max=(numA>numB)?numA>numC)?numA:numC):
(numB>numC)?numB:numC);
^

Solution.java:7: error: ';' expected
max=(numA>numB)?numA>numC)?numA:numC):
(numB>numC)?numB:numC);
^

Solution.java:10: error: '.class' expected

```
result=(max*int mid);
^
Solution.java:10: error: ';' expected
result=(max*int mid);
^
8 errors
```

Structural Vulnerabilities and Errors

There are no errors in the candidate's code.

Compilation Statistics

2

Total attempts

0

Successful

2

Compilation errors

0

Sample failed

0

Timed out

0

Runtime errors

Response time:

00:02:36

Average time taken between two compile attempts:

00:01:18

Average test case pass percentage per compile:

0%

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 3 (Language: Java)

The function/method ***drawPrintPattern*** accepts *num*, an integer.

The function/method ***drawPrintPattern*** prints the first *num* lines of the pattern shown below.

For example, if *num* = 3, the pattern should be:

```
1 1
```

```

+ +
1 1 1 1
1 1 1 1 1 1

```

The function/method **drawPrintPattern** compiles successfully but fails to get the desired result for some test cases due to incorrect implementation of the function/method. Your task is to fix the code so that it passes all the test cases.

Scores

Final Code Submitted

Compilation Status: Pass

```

1 // You can print the values to stdout for debugging
2 public class Solution
3 {
4     void drawPrintPattern(int num)
5     {
6         int i,j,print = 1;
7         for(i=1;i<=num;i++)
8         {
9             for(j=1;j<=2*i;j++);
10            {
11                System.out.print(print+" ");
12            }
13            System.out.println();
14        }
15    }
16
17 }
18

```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilities and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: 12.5%

Total score

1/8

0%

Basic(0/7)

0%

Advance(0/0)

100%

Edge(1/1)

Compilation Statistics

1

Total attempts

1

Successful

0

Compilation errors

1

Sample failed

0

Timed out

0

Runtime errors

Response time:

00:01:38

Average time taken between two compile attempts:

00:01:38

Average test case pass percentage per compile:

0%

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 4 (Language: Java)

The function/method ***sameElementCount*** returns an integer representing the number of elements of the input list which are even numbers and equal to the element to its right. For example, if the input list is [4 4 4 1 8 4 1 1 2 2] then the function/method should return the output '3' as it has three similar groups i.e, (4, 4), (4, 4), (2, 2).

The function/method ***sameElementCount*** accepts two arguments - *size*, an integer representing the size of the input list and *inputList*, a list of integers representing the input list.

The function/method compiles successfully but fails to return the desired result for some test cases due to incorrect implementation of the function/method ***sameElementCount***. Your task is to fix the code so that it passes all the test cases.

Note:

In a list, an element at index *i* is considered to be on the left of index *i+1* and to the right of index *i-1*. The last element of the input list does not have any element next to it which makes it incapable to satisfy the second condition and hence should not be counted.

Scores

Final Code Submitted

Compilation Status: Pass

```
1 // You can print the values to stdout for debugging
2 class Solution
3 {
4     int sameElementCount(int size, int[] inputList)
5     {
6         int count =0;
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

```

7   for(int i=0;i<size-1;i++)
8   {
9       if((inputList[i]%2==0)&&(inputList[i]==inputList[i++]))
10          count++;
11
12   }
13   return count;
14 }
15 }

```

Best case code:

 *N represents

Errors/Warnings

 There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

 There are no errors in the candidate's code.

Test Case Execution
Passed TC: 50%

Total score

 4/8

43%
 Basic(3/7)

0%
 Advance(0/0)

100%
 Edge(1/1)

Compilation Statistics

3

 Total attempts

3

 Successful

0

 Compilation errors

3

 Sample failed

0

 Timed out

0

 Runtime errors

Response time:
 00:03:01

Average time taken between two compile attempts:
 00:01:00

Average test case pass percentage per compile:
 8.3%

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 5 (Language: Java)

The function/method **manchester** print space-separated integers with the following property: for each element in the input list *arr*, if the bit *arr[i]* is the same as *arr[i-1]*, then the element of the output list is 0. If they are different, then its 1. For the first bit in the input list, assume its previous bit to be 0. This encoding is stored in a new list.

The function/method **manchester** accepts two arguments - *len*, an integer representing the length of the list and *arr* and *arr*, a list of integers, respectively. Each element of *arr* represents a bit - 0 or 1

For example - if *arr* is {0 1 0 0 1 1 1 0}, the function/method should print an list {0 1 1 0 1 0 0 1}.

The function/method compiles successfully but fails to print the desired result for some test cases due to logical errors. Your task is to fix the code so that it passes all the test cases.

Scores

Final Code Submitted

Compilation Status: Pass

```
1 // You can print the values to stdout for debugging
2 class Solution
3 {
4     public void manchester(int len, int[] arr)
5     {
6         int res[] = new int[len];
7         boolean result;
8         res [0] = arr [0];
9         for(int i = 1 ; i < len; i ++ )
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

```

10  {
11      result = ( arr[i] == arr[i-1] );
12      res[i] = (result) ? 1 : 0;
13  }
14  for(int i = 0; i < len; i++)
15      System.out.print(res[i]+" ");
16  }
17 }

```

Errors/Warnings

There are no errors in the candidate's code.

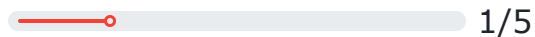
Structural Vulnerabilities and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: 20%

Total score



50%

Basic(1/2)

0%

Advance(0/3)

0%

Edge(0/0)

Compilation Statistics

3

Total attempts

3

Successful

0

Compilation errors

2

Sample failed

0

Timed out

1

Runtime errors

Response time:

00:02:49

Average time taken between two compile attempts:

00:00:56

Average test case pass percentage per compile:

0%

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 6 (Language: Java)

You are given a predefined structure/class **Point** and also a collection of related functions/methods that can be used to perform some basic operations on the structure.

The function/method **isRightTriangle** returns an integer '1', if the points make a right-angled triangle otherwise return '0'.

The function/method **isRightTriangle** accepts three points - *P1*, *P2*, *P3* representing the input points.

You are supposed to use the given function to complete the code of the function/method **isRightTriangle** so that it passes all test cases.

Helper Description

The following class is used to represent a point and is already implemented in the default code (Do not write this definition again in your code):

```
public class Point
{
    public int x, y;

    public Point()
    {}

    public Point(int x1, int y1)
    {
        x = x1 ;
        y = y1 ;
    }

    public double calculateDistance(Point P)
    {
        /*Return the euclidean distance between two input points.

        This can be called as -

        * If P1 and P2 are two points then -

        * P1.calculateDistance(P2);*/
    }
}
```

Scores

Final Code Submitted

Compilation Status: Fail

```
1 // You can print the values to stdout for debugging
2 public class Solution
3 {
4     int isRightTriangle(Point P1, Point P2, Point P3)
5     {
6         // write your code here
7     }
8 }
9
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

Solution.java:7: error: missing return statement
 }
 ^
 1 error

Structural Vulnerabilities and Errors

There are no errors in the candidate's code.

Compilation Statistics

0

Total attempts

0

Successful

0

Compilation errors

0

Sample failed

0

Timed out

0

Runtime errors

Response time:

00:00:05

Average time taken between two compile attempts:

00:00:00

Average test case pass percentage per compile:

0%

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 7 (Language: Java)

The function/method **countOccurrence** return an integer representing the count of occurrences of given value in the input list.

The function/method **countOccurrence** accepts three arguments - *len*, an integer representing the size of the input list, *value*, an integer representing the given value and *arr*, a list of integers, representing the input list.

The function/method **countOccurrence** compiles successfully but fails to return the desired result for some test cases due to logical errors. Your task is to fix the code so that it passes all the test cases.

Scores

Final Code Submitted

Compilation Status: Pass

```
1 // You can print the values to stdout for debugging
2 class Solution
3 {
4     public int countOccurrence( int len, int value, int arr[])
5     {
6         int i = 0 , count = 0;
7         while( i < len )
8         {
9             if( arr[i] == value )
10                count += 1;
11         }
12         return count;
13     }
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

14 }

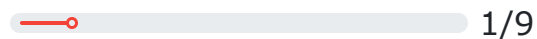
Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: 11.11%

Total score



0%

Basic(0/3)

0%

Advance(0/5)

100%

Edge(1/1)

Compilation Statistics

0

Total attempts

0

Successful

0

Compilation errors

1

Sample failed

1

Timed out

0

Runtime errors

Response time:

00:00:14

Average time taken between two compile attempts:

00:00:00

Average test case pass percentage per compile:

11.1%

i Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

i Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

WriteX - Essay Writing



5 / 100

CEFR: A1

Question

Nowadays, many youngsters constantly look for job opportunities abroad and in the long term want to settle there.

In your view, what are the reasons for the same? Do you wish to settle abroad or stay back in India? Substantiate your response with reasons.

Scores

Content Score

Grammar Score



5 / 100



5 / 100

Response

The Youngsters are looking the job in the abroad for more salary in the company and the company can offer the many opportunities to the employee and once the youngsters can get the job in aboard and they cannot return to the native and the abroad can youngsters are living there is not tax and the less expensive and the many company is offer to the employee to work in abroad and the employee need to good communication to any one and the employee need more experience in the work and the company give more salary to employee and they need to work at less time and the employee live in india and work at min 8 hours and the work more time and the salary is less .

Error Summary

	Spelling	4
	White Space	1
	Style	0
	Grammar	13
	Typographical	2

Essay Statistics

128

Total words

1

Total sentences

128

Average sentence length

49

Total unique words

69

Total stop words

Error Details

Spelling

...re living there is not tax and the less expensive and the many company is offer to the em...

Possible spelling mistake found

need to good communication to any one and the employee need

Possible spelling mistake found. Consider replacing the highlighted text with: 'anyone'.

...n to any one and the employee need more experience in the work and the company give more s...

Possible spelling mistake found

...k at less time and the employee live in india and work at min 8 hours and the work mo...

Possible spelling mistake found

White Space

...he work more time and the salary is less .

Don't put a space before the full stop

Grammar

The Youngsters are looking the job in the abroad for more salary in the company and the company can offer the many opportunities to the employee and once the youngsters can get the job in aboard and they cannot return to the native and the abroad can youngsters are living there is not tax and the less expensive and the many company is offer to the employee to work in abroad and the employee need to good communication to any one and the employee need more experience in the work and the company give more salary to employee and they need to work at less time and the employee live in india and work at min 8 hours and the work more time and the salary is less .

Possible grammar error found. Consider removing "The" from here.

The Youngsters are looking the job in the abroad for more salary in the company and the company can offer the many opportunities to the employee and once the youngsters can get the job in aboard and they cannot return to the native and the abroad can youngsters are living there is not tax and the less expensive and the many company is offer to the employee to work in abroad and the employee need to good communication to any one and the employee need more experience in the work and the company give more salary to employee and they need to work at less time and the employee live in india and work at min 8 hours and the work more time and the salary is less .

Possible grammar error found. Consider replacing it with "a".

The Youngsters are looking the job in the abroad for more salary in the company and the company can offer the many opportunities to the employee and once the youngsters can get the job in aboard and they cannot return to the native and the abroad can youngsters are living there is not tax and the less expensive and the many company is offer to the employee to work in abroad and the employee need to good communication to any one and the employee need more experience in the work and the company give more salary to employee and they need to work at less time and the employee live in india and work at min 8 hours and the work more time and the salary is less .

Possible grammar error found. Consider inserting "for" over here.

The Youngsters are looking the job in the abroad for more salary in the company and the company can offer the many opportunities to the employee and once the youngsters can get the job in aboard and they cannot return to the native and the abroad can youngsters are living there is not tax and the less expensive and the many company is offer to the employee to work in abroad and the employee need to good communication to any one and the employee need more experience in the work and the company give more salary to employee and they need to work at less time and the employee live in india and work at min 8 hours and the work more time and the salary is less .

Possible grammar error found. Consider replacing it with "a".

The Youngsters are looking the job in the abroad for more salary in the company and the company can offer the many opportunities to the employee and once the youngsters can get the job in aboard and they cannot return to the native and the abroad can youngsters are living there is not tax and the less expensive and the many company is offer to the employee to work in abroad and the employee need to good communication to any one and the employee need more experience in the work and the company give more salary to employee and they need to work at less time and the employee live in india and work at min 8 hours and the work more time and the salary is less .

Possible grammar error found. Consider removing "the" from here.

The Youngsters are looking the job in the abroad for more salary in the company and the company can offer the many opportunities to the employee and once the youngsters can get the job in aboard and they cannot return to the native and the abroad can youngsters are living there is not tax and the less expensive and the many company is offer to the employee to work in abroad and the employee need to good communication to any one and the employee need more experience in the work and the company give more salary to employee and they need to work at less time and the employee live in india and work at min 8 hours and the work more time and the salary is less .

Possible grammar error found. Consider replacing it with "employees".

The Youngsters are looking the job in the abroad for more salary in the company and the company can offer the many opportunities to the employee and once the youngsters can get the job in aboard and they cannot return to the native and the abroad can youngsters are living there is not tax and the less expensive and the many company is offer to the employee to work in abroad and the employee need to good communication to any one and the employee need more experience in the work and the company give more salary to employee and they need to work at less time and the employee live in india and work at min 8 hours and the work more time and the salary is less .

Possible grammar error found. Consider replacing it with "a".

The Youngsters are looking the job in the abroad for more salary in the company and the company can offer the many opportunities to the employee and once the youngsters can get the job in aboard and they cannot return to the native and the abroad can youngsters are living there is not tax and the less expensive and the many company is offer to the employee to work in abroad and the employee need to good communication to any one and the employee need more experience in the work and the company give more salary to employee and they need to work at less time and the employee live in india and work at min 8 hours and the work more time and the salary is less .

Possible grammar error found. Consider replacing it with "can".

The Youngsters are looking the job in the abroad for more salary in the company and the company can offer the many opportunities to the employee and once the youngsters can get the job in aboard and they cannot return to the native and the abroad can youngsters are living there is not tax and the less expensive and the many company is offer to the employee to work in abroad and the employee need to good communication to any one and the employee need more experience in the work and the company give more salary to employee and they need to work at less time and the employee live in india and work at min 8 hours and the work more time and the salary is less .

Possible grammar error found. Consider replacing it with "their".

The Youngsters are looking the job in the abroad for more salary in the company and the company can offer the many opportunities to the employee and once the youngsters can get the job in aboard and they cannot return to the native and the abroad can youngsters are living there is not tax and the less expensive and the many company is offer to the employee to work in abroad and the employee need to good communication to any one and the employee need more experience in the work and the company give more salary to employee and they need to work at less time and the employee live in india and work at min 8 hours and the work more time and the salary is less .

Possible grammar error found. Consider removing "the" from here.

The Youngsters are looking the job in the abroad for more salary in the company and the company can offer the many opportunities to the employee and once the youngsters can get the job in aboard and they cannot return to the native and the abroad can youngsters are living there is not tax and the less expensive and the many company is offer to the employee to work in abroad and the employee need to good communication to any one and the employee need more experience in the work and the company give more salary to employee and they need to work at less time and the employee live in india and work at min 8 hours and the work more time and the salary is less .

Possible grammar error found. Consider replacing it with "people".

The Youngsters are looking the job in the abroad for more salary in the company and the company can offer the many opportunities to the employee and once the youngsters can get the job in aboard and they cannot return to the native and the abroad can youngsters are living there is not tax and the less expensive and the many company is offer to the employee to work in abroad and the employee need to good communication to any one and the employee need more experience in the work and the company give more salary to employee and they need to work at less time and the employee live in india and work at min 8 hours and the work more time and the salary is less .

Possible grammar error found. Consider replacing it with "live".

The Youngsters are looking the job in the abroad for more salary in the company and the company can offer the many opportunities to the employee and once the youngsters can get the job in aboard and they cannot return to the native and the abroad can youngsters are living there is not tax and the less expensive and the many company is offer to the employee to work in abroad and the employee need to good communication to any one and the employee need more experience in the work and the company give more salary to employee and they need to work at less time and the employee live in india and work at min 8 hours and the work more time and the salary is less .

Possible grammar error found. Consider removing "the" from here.

Typographical

... once the youngsters can get the job in aboard and they cannot return to the native and th...

Use a comma before 'and' if it connects two independent clauses (unless they are closely connected and short).

...ork and the company give more salary to employee and they need to work at less time and the ...

Use a comma before 'and' if it connects two independent clauses (unless they are closely connected and short).

4 | Learning Resources

English Comprehension

[Learn about english in text and conversation](#)



[Practice your reading skills](#)



[Learn about the rules of english grammar](#)



Logical Ability

[Learn about generalizing unknown trends](#)



[Test your application of inductive logic!](#)



[Play Sudoku and develop your skills of deduction!](#)



Quantitative Ability (Advanced)

[Learn about multiplication and division in the real world contexts](#)



[Learn about multiplication and division](#)



[Learn about mathematic operations](#)



Icon Index



Free Tutorial



Paid Tutorial



Youtube Video



Web Source



Wikipedia



Text Tutorial



Video Tutorial



Google Playstore